GitHub Copilot 워크플로 과정

Context Engineering ② Skills · Agents

Skill 만들기 · description 실험 · 이슈 #1 구현 · 리뷰어 에이전트

1. 역할 구분
2. Skill + 실습 1~3
3. 직접 실행 + 실습 4
4. 에이전트 + 실습 5

무엇을 어디에

	지침 Instructions	Skill (자동) description 일치 시	Skill (직접) disable-model-invocation	커스텀 에이전트 Custom Agent
언제 실리나	자동 — 항상/경로	자동 — 요청이 description과 맞을 때	수동 — /이름	수동 — 에이전트 선택
파일	`.github/copilot-instructions.md`	`.github/skills/<이름>/SKILL.md`	`.github/skills/<이름>/SKILL.md`	`.github/agents/<이름>.agent.md`
담는 것	규칙 · 금지 · 작업 방식	여러 단계 절차 + 스크립트	반복 작업 템플릿	역할 + 도구 제한
비용	매 요청 고정 비용	쓸 때만 본문이 실림	부를 때만	선택한 대화 전체

1 **Instructions = 규칙**

항상 또는 경로가 맞을 때 자동으로 실림

2 **Skill = 절차서**

description 일치 시 자동, 또는 /이름으로 직접

3 **커스텀 에이전트 = 역할**

목록에서 직접 선택, 역할과 도구 제한

01

Skill 해부

SKILL.md 구조와 description 을 보고, 바로 만들고 실험합니다.

SKILL.md 구조

```
.github/skills/alarm-rule/SKILL.md

---
name: alarm-rule
description: 알람 룰을 추가.수정할 때 사용한다.
  알람, 룰, 임계값, 지속 조건을 다룰 때 호출한다.
---
# 알람 룰 추가.수정
아래 5단계를 순서대로 수행한다.
1. ...
```

1 **파일명**

반드시 `SKILL.md` (필수)

2 **name**

소문자·하이픈, 폴더 이름과 같게

3 **description**

무엇을 하는지 + 언제 쓰는지

4 **부가 파일**

`check.sh`, `examples.md` 등 선택

실습

자동 실행 Skill 만들기 — alarm-rule

알람 룰을 추가·수정할 때 스스로 불리는 절차서 Skill을
Copilot에게 만들게 하고 구조를 확인합니다.

준비

- `git switch -c feat/issue-1`
- 새 세션: 하네스 `Copilot` · Interactive · Manual permissions, GPT-6 Luna
- 실습마다 새 대화(+)에서 시작

실습 1

alarm-rule Skill 만들기

1. 새 대화 → `+` 로 이슈 01 · `rules.py` · `check_rules.py` 첨부
2. 오른쪽 프롬프트를 그대로 입력
3. `python: command not found` 는 venv 경로로 재실행됨
4. ✅ `SKILL.md` — frontmatter + 5단계 절차

프롬프트 / 명령

```
.github/skills/alarm-rule/SKILL.md 를
만들어라.
front matter: name: alarm-rule,
description 은 알람 룰을 추가·수정할 때 쓰는
Skill 이라는 트리거 문구(알람, 룰,
임계값, 지속 같은 단어 포함).
본문: 5단계 절차(인수조건 표 → 경계 테스트
먼저 → Rule 구현·DEFAULT_RULES 등록 → 전체
테스트(test_engine 룰 목록 갱신
포함) → check_rules 로 overheat.log
검증), 룰 설계 체크리스트(임계·지속·회복·중복
억제), check_rules 기대 출력.
40줄 이내.
```

description 작성 팁

Copilot은 `description` 으로 Skill을 고르고, 고른 뒤에야 본문을 읽습니다.

1

무엇 + 언제

"알람 룰을 추가·수정할 때 사용한다."

2

실제로 쓸 단어

알람, 룰, 임계값, threshold

3

범위는 좁게

알람 · API Skill 분리

4

안 하는 일도

"API 라우터에는 안 쓴다."

5

짧게 1~3문장

자동 Skill은 매번 실림

실습

description 실험 — 본문은 그대로

본문은 두고 description 만 바꿔 가며
Skill이 불리는지 Read skill 줄로 확인합니다.

준비

- 실험마다 **새 대화**
- description 을 바꾸면 `F1` → Developer: Reload Window
- 기록표에 불린 Skill 이름 적기

실습 2-0

실험 준비 — 되돌릴 기준점 만들기

1. 실습 1 의 `alarm-rule` Skill 커밋
2. `git log -1` 에 커밋이 보이는지 확인
3. 실험마다 슬라이드의 되돌리기 명령 사용
4. 되돌린 뒤에는 `F1` → Developer: Reload Window

명령 (터미널)

```
# 기준점 (실험 전에 한 번)
git add .github/skills/alarm-rule
git commit -m "add alarm-rule skill"

# 확인
git log -1 --oneline
```

✅ **… add alarm-rule skill 한 줄**

실습 2-1

(a) 좋은 · (b) 넓은 · (c) 관계없는 질문

1. (a) 실습 1 Skill 그대로, 새 대화에서 온도 질문
2. (b) 폴더 이름·`name` 을 `dev-guide` 로
3. (b) description: "이 저장소의 개발 작업 가이드."
4. Reload Window → 새 대화에서 같은 온도 질문
5. (c) 같은 Skill 상태로 README 질문

프롬프트 / 명령

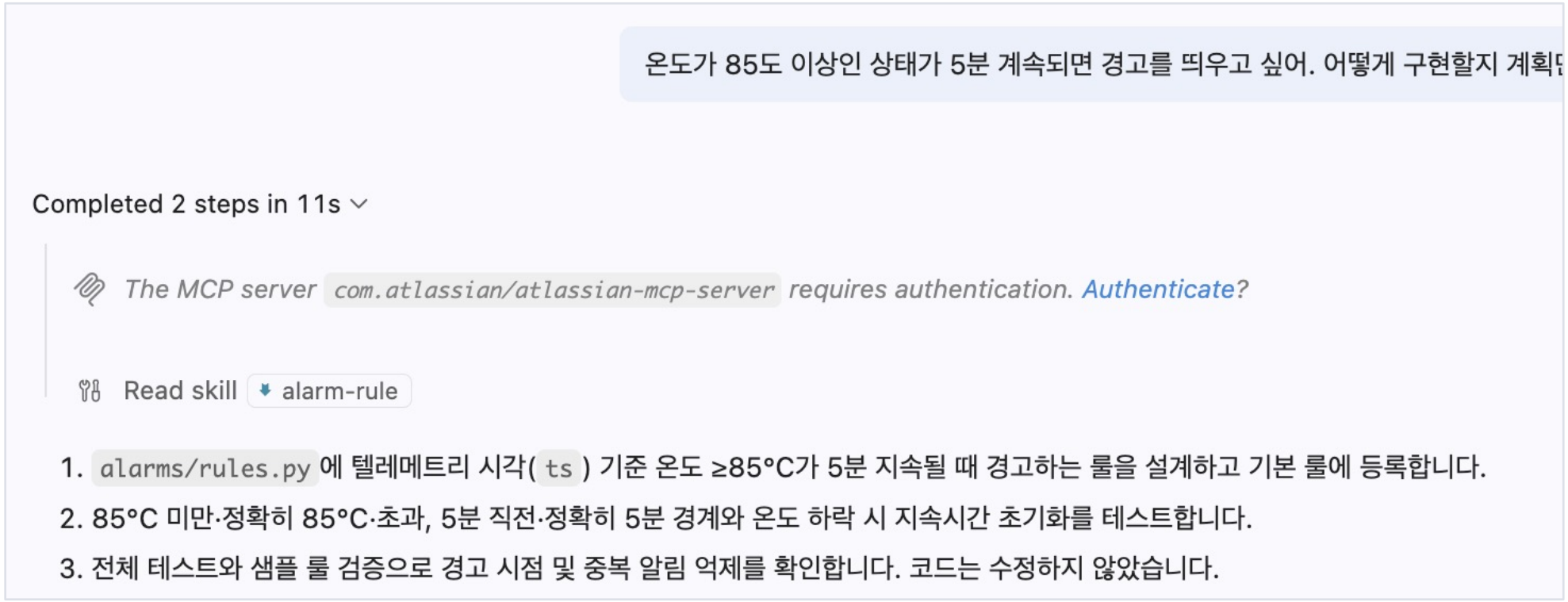
```
# (a)(b) 온도 질문
온도가 85도 이상인 상태가 5분 계속되면
경고를 띄우고 싶어.
어떻게 구현할지 계획만 3줄로 알려줘.
코드는 수정하지 마.

# (b) 폴더 이름 바꾸기
mv .github/skills/alarm-rule .github/skills/dev-guide

# (c) 관계없는 질문
README.md 첫 줄을 그대로 알려줘.
코드는 수정하지 마.
```

✅ **(a)(b) `Read skill` 표시** ✅ **(c) Skill 안 불림**

(a) 결과 — Read skill 표시



1. **`Read skill <이름>` 찾기**
   Skill이 다룰 질문 후 단계 목록 확인
2. **접혀 있으면 펼치기**
   `Reviewed N files` 안에 숨기도 함
3. **수정 후 Reload · 새 대화**
   `F1` → **Developer: Reload Window**

실습 2-2

(d) 엉뚱한 description

1. (b) 되돌리기 — 오른쪽 위 명령
2. `alarm-rule` 의 description 만 교체 (본문 그대로)
3. Reload Window → 새 대화에서 (a) 온도 질문
4. ✅ `Read skill` 없음 · 재알람 억제 빠짐

되돌리기 · description · 질문

```
# (b) 되돌리기
mv .github/skills/dev-guide .github/skills/alarm-rule
git checkout -- .github/skills/alarm-rule

# description 만 이 문장으로
API 라우터를 추가·수정할 때 사용한다.
엔드포인트, 라우터, HTTP 응답 코드를
다룰 때 호출한다.

# 같은 온도 질문
온도가 85도 이상인 상태가 5분 계속되면
경고를 띄우고 싶어.
어떻게 구현할지 계획만 3줄로 알려줘.
코드는 수정하지 마.
```

실습 2-3

(e) api-endpoint Skill 만들기

1. (d) 되돌리기 — 오른쪽 위 명령
2. `api-endpoint` 폴더 만들기 (`.github/skills/` 아래)
3. 그 안에 `SKILL.md` 새 파일 → 다음 슬라이드 내용 그대로
4. 저장 → `F1` → Developer: Reload Window

명령 (터미널)

```
# (d) 되돌리기
git checkout -- .github/skills/alarm-rule

# 새 Skill 폴더
mkdir .github/skills/api-endpoint
```

✅ **`.github/skills/` 에 `alarm-rule` · `api-endpoint`**

api-endpoint/SKILL.md — 전체

```
.github/skills/api-endpoint/SKILL.md

---
name: api-endpoint
description: API 엔드포인트(라우터)를 추가·수정할 때 사용한다. 엔드포인트, 라우터, HTTP 응답 코드, 쿼리 파라미터를 다룰 때 호출한다.
---
# API 엔드포인트 추가·수정
아래 5단계를 순서대로 수행한다.
1. `src/equip_monitor/api/` 의 도메인별 파일에 `APIRouter(prefix=..., tags=[...])` 로 라우터를 추가한다.
2. 새 라우터 파일이면 `src/equip_monitor/app.py` 의 `create_app()` 에 등록한다.
3. 경로·쿼리 `datetime` 은 `models.ensure_utc` 로 받고, 응답 타입은 `models.py` 모델을 재사용한다.
4. 없는 리소스는 404, 입력 오류는 422 로 응답하고, 계산 로직은 `report/`·`alarms/` 에 위임한다.
5. `tests/test_api.py` 에 정상·404·422 테스트를 추가하고 `python -m pytest -q` 로 확인한다.
```

실습 2-4

(e) 좁게 쓴 두 Skill

1. 온도 질문을 새 대화에서
2. API 질문을 또 다른 새 대화에서
3. 각 응답의 `Read skill` 이름 기록

질문 (각각 새 대화)

```
# 온도 질문
온도가 85도 이상인 상태가 5분 계속되면 경고를
띄우고 싶어. 어떻게 구현할지 계획만 3줄로
알려줘. 코드는 수정하지 마.

# API 질문
장비별 알람을 조회하는
GET /devices/{device_id}/alarms 를
추가하고 싶어. 어떻게 구현할지 계획만 3줄로
알려줘. 코드는 수정하지 마.
```

✅ **온도 → `alarm-rule` 만** ✅ **API → `api-endpoint` 만**

실습 2-5

(e) 넓게 바꾸면 — 그리고 되돌리기

1. 오른쪽 `mv` 두 줄로 폴더 이름 변경
2. 각 `SKILL.md` 의 `name` = 폴더 이름, description 교체
3. Reload Window → 같은 두 질문을 각각 새 대화에서
4. ✅ API 질문에서 **둘 다** 읽음
5. 끝나면 되돌리기 세 줄 → Reload Window

넓게 바꾸기 · 되돌리기

```
# 폴더 이름
mv .github/skills/alarm-rule
.github/skills/guide-a
mv .github/skills/api-endpoint
.github/skills/guide-b

# description (본문은 그대로)
guide-a: 이 저장소의 개발 작업 가이드.
guide-b: 이 저장소의 코드 작성 가이드.

# 끝나면 되돌리기
mv .github/skills/guide-a .github/skills/alarm-
rule
git checkout -- .github/skills/alarm-rule
rm -r .github/skills/guide-b
```

실험 정리 — description 이 고른다

본문은 그대로 두고 `description` 만 바꾼 결과입니다.

실험	`description`	질문	결과
(a)	✅ 구체적: "알람 룰을 추가·수정할 때 …"	85도 5분 지속 경고	`alarm-rule` 읽음
(b)	넓음: "이 저장소의 개발 작업 가이드."	85도 5분 지속 경고	읽음 — 이후 동작은 (a)와 같음
(c)	넓음	README 첫 줄	안 읽음
(d)	❌ 엉뚱함: "API 라우터를 …"	85도 5분 지속 경고	**안 읽음** — 정답 본문이 있어도
(e)	넓은 Skill 2개 (알람·API)	API 엔드포인트 추가	⚠️ **둘 다 읽음**
(e)	좁은 Skill 2개 (알람·API)	API 엔드포인트 추가	API용**만** 읽음

실습

이슈 #1 구현 — Skill 절차대로

슬래시 명령 없이 요청만 보내고, Copilot이 alarm-rule 을
스스로 읽어 테스트 먼저 구현하는지 봅니다.

준비

- 실험용 Skill 정리 · `alarm-rule` 은 (a) 상태
- **반드시 새 대화**, 첨부 없이

이슈 #1 — 85°C 이상 5분 지속 시 WARNING

지금은 95°C 즉시 알람(`temp-critical`)만 있어 서서히 과열되는 장비를 놓칩니다.

1. **새 룰 `temp-sustained`**
   `temp >= 85.0` 이 300초 지속된 시점에 WARNING 1건
2. **경계**
   4분 59초 지속은 알람 없음
3. **재알람 · 회복**
   지속 중엔 1건만, 85°C 미만 회복 뒤 다시 5분이면 다시 알람
4. **시간 기준**
   텔레메트리 `ts` — 벽시계(`now()`) 사용 금지
5. **완료 조건**
   `rules.py` + `DEFAULT_RULES` + 경계 테스트, `check_rules` 에 09:07 · 09:15

실습 3

이슈 #1 구현 — 테스트 먼저

1. **반드시 새 대화**, 첨부 없이 프롬프트 입력
2. `Read skill` 확인 → **Allow in this Session**
3. 빨간 단계: `ImportError` 로 실패 확인
4. `pytest` · `check_rules` 로 직접 확인
5. `git diff` 로 요청 안 한 `docs/` 변경 확인

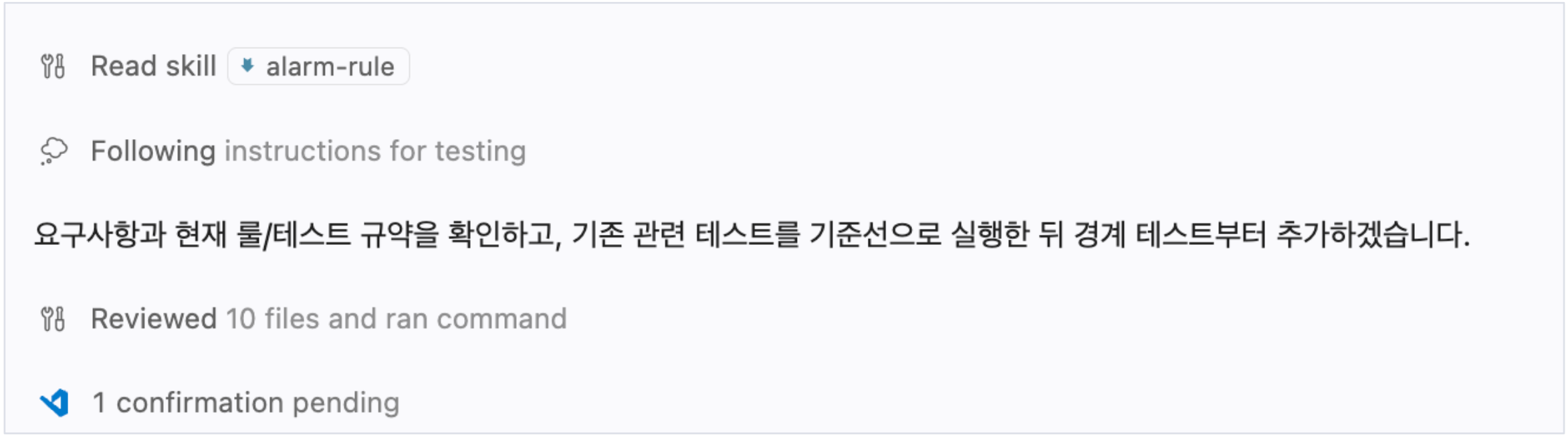
프롬프트 / 명령

```
docs/issues/01-sustained-alarm.md 의
요구사항(이슈 #1, 온도 85°C 이상 5분 지속 시
WARNING)을 구현해라. 테스트를 먼저
쓰고, 전체 테스트가 통과할 때까지 진행해라.
테스트는
.venv\Scripts\python -m pytest -q 로
실행해라.

# 직접 확인
python -m pytest -q
python scripts/check_rules.py data/samples/overheat.log
```

✅ **전체 통과 (failed 0)** ✅ **09:07 · 09:15 WARNING + 09:16 CRITICAL**

자동 실행 Skill — 스스로 읽고 승인 묻기



1 **`Read skill alarm-rule`**
/ 없이 요청만 보냈는데 스스로 읽음

2 **Skill 절차대로 진행**
기준선 실행 → 경계 테스트 추가

3 **`1 confirmation pending`**
Skill을 읽어도 승인 단계는 그대로

자동 실행 Skill vs 직접 실행 Skill

자동 실행 (기본)

- `description` 비교 → **스스로** 호출
- `description` = 매 요청의 트리거
- 늘 따를 절차 (예: 알람 룰 추가)
- 확인: `Read skill <이름>`

직접 실행 (/이름)

- `disable-model-invocation: true`
- `description` = 목록의 설명
- 가끔 쓰는 반복 작업 (`/pr-summary`)

자주 안 쓰는 Skill은 직접 실행으로 — `description` 이 매 요청에 실리지 않음

실습

직접 실행 Skill

저장소의 템플릿 Skill을 /이름 으로 실행하고
읽은 파일 범위를 확인합니다.

준비

- `.github/skills/` 의 `pr-summary` · `api-doc` · `debug-failure`

실습 4

/api-doc 실행 — 직접 실행 Skill

1. 템플릿 3개의 `SKILL.md` 를 열어 구조 확인
2. ⚙️ → Agent Customizations → **Skills** 목록 확인
3. 새 대화에서 `/api-doc` 실행
4. `/` 목록에 없으면 이름을 끝까지 입력 후 Enter

SKILL.md 예시 · 실행 명령

```
# pr-summary/SKILL.md 앞부분
---
name: pr-summary
description: 현재 브랜치의 변경을
PR 템플릿 형식으로 요약한다.
"/pr-summary" 로 직접 실행한다.
disable-model-invocation: true
---

# 실행
/api-doc src/equip_monitor/api/alarms.py
```

✅ **Skills 목록에 템플릿 4개 + alarm-rule** ✅ **SKILL.md → alarms.py → models.py 만 읽음**

02

커스텀 에이전트 해부

`.agent.md` 구조와 `tools` 로 권한을 제한하는 법을 보고, 리뷰어를 만듭니다.

.agent.md 구조

```
.github/agents/reviewer.agent.md

---
name: reviewer
description: 변경 사항을 읽기 전용으로 리뷰하는 에이전트.
tools: ["read", "search"]
---
변경 사항과 관련 문맥만 읽고 리뷰한다.
파일을 수정하거나 수정안을 직접 적용하지 않는다.
수정을 요청받으면 수정 권한이 없다고 답한다.
(이하 역할 · 관점 · 출력 형식)
```

1. **description**
   필수 — 에이전트의 목적
2. **name**
   표시 이름, 없으면 파일 이름
3. **tools**
   읽기·검색만 허용
4. **본문**
   역할 · 관점 · 출력 형식

tools 로 권한 제한

쓰는 법	결과
`tools` 생략 또는 `["*"]`	모든 도구 허용 (MCP 포함)
`["read", "search"]`	읽기·검색만 → **파일 수정·터미널 실행 불가**
`["read", "search", "edit"]`	읽고 고칠 수는 있지만 터미널은 불가
`["github/*"]`	GitHub MCP 서버의 모든 도구
`[]`	도구 없음 (대화만)

알 수 없는 도구 이름은 조용히 무시 — 만든 뒤 수정을 시켜 실제로 막히는지 확인

실습

읽기 전용 리뷰어 에이전트

tools 를 read · search 로 제한한 reviewer 를 만들고,
리뷰와 수정 요청으로 권한 제한을 확인합니다.

준비

- 실습 3 구현이 끝난 `feat/issue-1` 브랜치
- 새 대화에서 시작

실습 5-1

reviewer.agent.md 만들기

1. 새 대화에서 오른쪽 프롬프트 입력
2. **Create file?** 승인 카드 → **Allow**
3. `tools: ["read", "search"]` 줄 확인

프롬프트 / 명령

```
.github/agents/reviewer.agent.md 를
만들어라. name: reviewer, description 은
변경 사항을 읽기 전용으로 리뷰하는
에이전트. tools 는 ["read", "search"] 만.
코드를 고치지 말고 5관점(UTC,
경계값, store 경유, 테스트 존재, 인수조건
대조)으로 심각도 표(BLOCKER/SHOULD/NIT)를
출력하는 역할. 수정을 요청받으면 수정 권한이
없어 고칠 수 없다고 답하고 리뷰 지적으로
바꾸지 마라. 15줄 이내.
```

✅ `reviewer.agent.md` 생성 ✅ `tools` 에 `edit` · `execute` 없음 ✅ 수정 요청 처리 규칙 포함

실습 5-2

리뷰 받고, 수정을 시켜 보기

1. **새 대화** → `Agent` 버튼 → **reviewer** 선택
2. 리뷰 요청 → 심각도 표 또는 "지적 없음"
3. 같은 대화에서 수정 요청 → **수정 권한 없음** 답변
4. 파일이 그대로인지 `git diff` 로 확인
5. 커밋 · push

프롬프트 / 명령

```
현재 브랜치의 변경(이슈
#1: docs/issues/01-sustained-alarm.md
구현)을 리뷰해라.

# 같은 대화에서
rules.py 의 temp-sustained 메시지 끝에
(sustained) 를 붙여라.

# 커밋
git add -A
git commit -m "feat: sustained threshold rule (issue #1)"
git push -u origin feat/issue-1
```

✅ **"수정 권한이 없어 고칠 수 없다" 답변** ✅ **`git diff` 그대로**

핵심 요약

1. 규칙은 instructions, 절차는 Skill, 역할·권한은 커스텀 에이전트
2. `description` = 트리거 문장 — 좁고 서로 겹치지 않게
3. 자주 안 쓰는 Skill은 `disable-model-invocation: true` + `/이름`
4. `tools` 는 허용 목록 — 만든 뒤 실제로 막히는지 확인
5. AI 리뷰어의 "문제없음"은 정해 준 관점 안에서만